

Observer

Refactoring en el mundo real — Sistema de inventario que notifica a múltiples servicios

Parámetro	Valor
Asignatura	Metodología de Sistemas II
Modalidad	Trabajo Práctico Individual
Tiempo estimado	2 a 3 horas
Lenguaje	Java, Python, C# o TypeScript
Restricción	Sin librerías externas

¿De qué trata este trabajo?

Esto no es un ejercicio teórico sobre definiciones. Es una simulación de lo que pasa todos los días en proyectos que llevan tiempo vivos: código que funciona, pero que ya nadie quiere tocar. Vas a recibir código real con problemas reales, y tu trabajo es entrar, entender qué está fallando, y refactorizarlo aplicando el patrón Observer.

Si entendés bien este patrón, tenés la lógica base para abordar cualquier patrón de comportamiento de la familia GoF por tu cuenta.

Restricción global

No uses librerías externas. Todo se resuelve con el lenguaje base que elijas. El objetivo es demostrar que entendés el patrón, no que encuentraste una librería que lo implementa.

¿Qué está pasando en el proyecto?

Tenemos un sistema de inventario. Cada vez que el stock de un producto baja, hay que notificar a tres sistemas: alertas por email, el panel de analytics y el módulo de reposición automática. El developer original lo hizo directo. Muy directo.

El código problemático

```
CLASE InventoryManager:
    CONSTRUCTOR:
        this.emailService    = NUEVO EmailAlertService()
        this.analytics        = NUEVO AnalyticsDashboard()
        this.replenishment    = NUEVO AutoReplenishment()
```

```

MÉTODO update_stock(productId, quantity):
    this.stock[productId] = quantity
    SI quantity < 10:
        this.emailService.send_low_stock_alert(productId, quantity)
        this.analytics.record_low_stock_event(productId)
        this.replenishment.trigger_order(productId, 100)

MÉTODO sell_product(productId, sold):
    this.stock[productId] -= sold
    SI this.stock[productId] < 10:
        this.emailService.send_low_stock_alert(productId, ...)
        this.analytics.record_low_stock_event(productId)
        this.replenishment.trigger_order(productId, 100)

```

El problema de fondo

El InventoryManager conoce por nombre a EmailAlertService, AnalyticsDashboard y AutoReplenishment. Si cualquiera de los tres cambia, o si querés agregar notificaciones push, tenés que abrir el InventoryManager. Que no tiene nada que ver con emails ni con analytics. Eso es acoplamiento.

¿Qué hace el patrón Observer?

El InventoryManager mantiene una lista genérica de "cosas que quieren saber cuando algo cambia". Cuando el stock baja, recorre la lista y avisa a cada uno. No sabe quiénes son — solo sabe que tienen un método onLowStock(). Cada observador decide qué hacer con el aviso.

Anclaje teórico

Observer (también llamado Observador, Publicación-Suscripción, Event-Subscriber, Listener) es un patrón de diseño de comportamiento que define un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

La estructura canónica del patrón identifica tres roles principales:

- **Notificador (Publisher):** Envía eventos de interés a otros objetos. Mantiene el campo de suscriptores y los métodos subscribe/unsubscribe/notifySubscribers.
- **Interfaz Suscriptora:** Declara la interfaz de notificación. Consiste típicamente en un único método update con parámetros de contexto.
- **Suscriptores Concretos:** Realizan acciones en respuesta a las notificaciones. Implementan la misma interfaz para que el notificador no esté acoplado a clases concretas.

En este ejercicio, InventoryManager juega el rol de Notificador, StockObserver es la Interfaz Suscriptora, y EmailAlertObserver/AnalyticsObserver/ReplenishObserver son los Suscriptores Concretos.

Diagrama UML — Antes vs Después

(Incluí en tu entrega un diagrama UML de la situación inicial y otro de la solución refactorizada.)

Pseudocódigo de la solución

1. La interfaz del observador

```
INTERFAZ StockObserver:  
    MÉTODO onLowStock(productId, quantity)
```

2. Cada servicio implementa la interfaz

```
CLASE EmailAlertObserver IMPLEMENTA StockObserver:  
    MÉTODO onLowStock(productId, qty):  
        enviar email "Stock bajo: " + productId  
  
CLASE AnalyticsObserver IMPLEMENTA StockObserver:  
    MÉTODO onLowStock(productId, qty):  
        registrar evento en el dashboard  
  
CLASE ReplenishObserver IMPLEMENTA StockObserver:  
    MÉTODO onLowStock(productId, qty):  
        crear orden de reposición por 100 unidades
```

3. El manager solo conoce la interfaz

```
CLASE InventoryManager:  
    CONSTRUCTOR:  
        this.observers = []          // lista vacía, sin nombres concretos  
  
    MÉTODO subscribe(observer: StockObserver):  
        this.observers.agregar(observer)  
  
    MÉTODO unsubscribe(observer: StockObserver):  
        this.observers.remove(observer)  
  
    MÉTODO notify(productId, qty):      // privado  
        PARA CADA observer EN this.observers:  
            INTENTAR:  
                observer.onLowStock(productId, qty)  
            SI FALLA:                    // DENTRO del loop  
                registrar el error  
            CONTINUAR con el siguiente  // no detiene la cadena  
  
    MÉTODO update_stock(productId, qty):  
        this.stock[productId] = qty  
        SI qty < 10:  
            this.notify(productId, qty)
```

Truco clave — dónde va el manejo de errores

El bloque de captura de errores debe estar DENTRO del loop, envolviendo cada llamada individual. Si está FUERA: el primer observer que falla corta toda la cadena. Si está DENTRO: cada observer es independiente — uno puede fallar sin afectar a los demás.

4. Código de prueba — el observer que falla

```
manager = NUEVO InventoryManager()
manager.subscribe(NUEVO EmailAlertObserver())
manager.subscribe(NUEVO AnalyticsObserver())

CLASE ObservadorRoto IMPLEMENTA StockObserver:
    MÉTODO onLowStock(productId, qty):
        LANZAR excepción "error de red simulado"

manager.subscribe(NUEVO ObservadorRoto())
manager.subscribe(NUEVO ReplenishObserver())

// Resultado esperado:
// email OK | ObservadorRoto falla y se registra | replenish OK
manager.update_stock("PROD-001", 5)
```

Tu misión

- Refactorizá el InventoryManager con Observer.
- El manager no puede referenciar a ningún observador concreto por nombre.
- Demostrá en código que un observer que falla no detiene a los demás.
- Agregá un observador nuevo (puede ser push, SMS, lo que quieras) para demostrar que el sistema es extensible sin tocar el manager.

Restricciones

Sin threads ni procesos adicionales — las notificaciones son síncronas. El test del observer roto es obligatorio en la entrega.

¿Qué espero ver en tu entrega?

Qué evalúo	Lo que estoy mirando
Diagrama UML	Antes y después. No tiene que ser perfecto — tiene que contar la historia a la primera.
Antes / Después	El código original junto al refactorizado. Sin el contraste no sé si entendiste qué estaba mal.
Justificación	Máximo 10 líneas explicando qué problema específico resolvió el patrón en este ejercicio. No la definición del libro.
Legibilidad	El código refactorizado debe entenderse sin explicaciones.
Test del observer roto	El observer que explota y los demás siguen corriendo. Tiene que estar en el código — no alcanza con mencionarlo.

Qué evalúo	Lo que estoy mirando
Restricciones	Todas respetadas. Librería externa = ejercicio no cuenta, aunque el código sea brillante.

Última cosa

Si tu solución no encaja del todo con el patrón "canónico" del libro, explicá por qué la elegiste así. Eso me dice más sobre cómo pensás que si copiaste la implementación de un tutorial.